Real-Time Parking Space Occupancy Detection Using Fine-Tuned YOLOv8 and Region-of-Interest Mapping

Sarvesh Bhamare
B.tech Cloud Techonology and Information Security
Ajeenkya DY Patil University
Pune, India
sarvesh.bhamare@adypu.edu.in

Sarvesh Bhamare
B.tech Cloud Techonology and Information Security
Ajeenkya DY Patil University
Pune, India
sarvesh.bhamare@adypu.edu.in

Sarvesh Bhamare
B.tech Cloud Techonology and Information Security
Ajeenkya DY Patil University
Pune, India
sarvesh.bhamare@adypu.edu.in

Abstract**—***The worsening urban problem of traffic congestion due to drivers' search for parking slots cannot be solved without addressing the inefficiency of parking spaces themselves. Traditional parking management systems, based on inductive loops, ultrasonic distance sensors, or radio-frequency identification tags, are costly and hard to deploy widely. This paper discusses a computer-vision parking slot occupancy detector using a fine-tuned YOLOv8 object detection algorithm augmented with a region-of-interest mapping slot occupancy classifier. A dataset of 2,412 labeled images of parking lots from Pklot public repository, openly accessible European traffic cameras, and the author's personal capture from the local commercial parking area was created to cover a wide variety of lighting, weather, and angle of observation. The model was trained for 60 epochs, starting with the pretrained YOLOv8s version, resulting in mAP@0.5 of 92.4%, precision of 93.1%, and recall of 90.8%. During inference, the slots are overlapped with the detected bounding boxes; a slot is considered occupied if their intersection over union score is above a predetermined value of 0.30. Overall, the model is run in real-time at a rate of roughly 28 fps, with each slot evaluated with accuracy of 94.7%. Compared to sensor-based methods, our solution offers an improvement in slot-classification accuracy of 6.2 percentage points. Our three main contributions to the problem are as follows: (1) A well-documented workflow to fine-tune YOLOv8 object detector in parking environments; (2) An ROI mapping slot occupancy classifier that does not require installation of any extra sensors; and (3) an analysis of failure modes—partial occlusions, shadows, and two-wheeled vehicles—that can help the practitioner deploy the proposed method realistically.*

Keywords*—YOLOv8, Smart Parking, Object Detection, Computer Vision, ROI Mapping, Real-Time Detection, Occupancy Classification, Deep Learning, Intelligent Transportation Systems.*

I. INTRODUCTION

For many motorists, locating a parking spot within an urban setting can be even more difficult than driving from one place to another. Researchers have established that within metropolitan environments during peak traffic times, up to 30–45 percent of all vehicles in circulation are circulating around looking for a parking spot [1]. Naturally, such a practice comes at a high cost, increasing overall fuel consumption, CO_2 emissions, causing additional stress for drivers, and even making a walking experience worse through urban centers. It appears that there is a mismatch between the amount of traffic in cities and their infrastructure's efficiency in directing automobiles to available spots.

The economics behind parking shortages are substantial as well. Within densely populated areas, the cost associated with driving around looking for a vacant spot is estimated at billions of dollars a year [1]. Furthermore, parking buildings are extremely expensive structures, meaning that underusing them due to inefficient communication regarding available parking spaces increases the return on investments for facility owners. As a result, over the years, there was significant development and implementation of technologies commonly referred to as 'smart parking'.

The traditional parking lot management methods can be divided into two types: the system based on physical sensing devices and manual management systems. The sensor-based parking systems, using such techniques as inductive loops placed on the pavement surface, ultrasonic proximity sensors fixed in every parking space, or magnetic field detectors installed below ground, provide acceptable accuracy at each parking slot. On the other hand, these solutions involve significant expenditures related to the implementation of such a parking system. Just one installation of an inductive loop may cost from a few thousand dollars per slot including the labor costs, while the maintenance activities associated with regular repairs, firmware updates, and wiring corrosion significantly increase the overall cost [2].

Computer vision offers a highly promising solution to this problem. Indeed, modern IP cameras are ubiquitous in almost all commercial parking lots for surveillance purposes. Hence, much of the necessary sensing equipment is already available. What remains is solving the problem of occupancy estimation based on those video feeds, i.e., performing real-time object detection without deploying additional hardware like bay sensors. Early solutions attempted to use hand-crafted features—texture analysis, edge detection, and background subtraction—to classify cars on images. Such an approach worked reasonably well under favorable circumstances but suffered when faced with adverse weather or poor lighting since hand-crafted features were not sufficiently expressive and therefore failed to generalize in such conditions [3].

However, the advent of deep neural network architectures brought about a paradigm shift in computer vision. Networks trained on extensive image databases managed to learn very versatile feature representations that exhibited surprising robustness to photometric and geometric variability—problems that earlier approaches could never solve. In this general context, one should mention the YOLO (You Only Look Once) series of models—an innovation that allowed for extremely fast object detection using single-stage detectors to predict bounding boxes and class scores in just a single network pass [4]. Since their inception in 2016 through YOLOv3, YOLOv4, YOLOv5, YOLOv7, and beyond, YOLO models kept pushing the envelope on accuracy and speed trade-offs in the field of object detection.The latest variant, YOLOv8 by Ultralytics released in early 2023, implemented multiple architectural improvements, including: an anchor-free detector, which obviates the necessity of manually designing anchor boxes; a decoupled architecture of YOLOv8 which splits classifier and detector branches allowing for independent optimization of each; and a novel backbone module—C2f—which exhibits better gradient propagation than previous modules [5]. All these architectural tweaks result in improved convergence behavior in fine-tuning on small domain-specific datasets—a practical advantage since labeled parking datasets are relatively small compared to benchmark datasets used for training YOLO variants.

This paper proposes a full system where a pre-trained YOLOv8 model is fine-tuned on parking-oriented imagery and a lightweight ROI overlap strategy is applied to produce slot-level

occupancy predictions from vehicle detections. Contributions can be classified along three axes. Firstly, an in-depth explanation is provided about the process of collecting, annotating and augmenting the training dataset to include samples reflecting the visual variations found in parking scenes, as well as tradeoffs and mitigations encountered during this process. Secondly, details of the fine-tuning pipeline are provided, together with hyperparameter values—namely batch size, learning rate schedule, epoch number and augmentation policy—chosen in order to obtain a mAP@0.5 of 92.4% on unseen data. Finally, a systematic exploration of failure modes is conducted. This includes an identification of the contexts in which the pipeline fails, the reasons for such failure, and possible mitigations. In practice, this means avoiding the common approach of overstating model performance by using top-line metrics, while pushing failure cases underfoot.

The rest of the paper is structured as follows. Section II provides background material on sensor-based and vision-based parking detection solutions. Section III covers the architecture, datasets, and inference methodology behind the system. Section IV reports quantitative results and qualitative insights gained. Section V highlights implications and limitations of this work. Section VI concludes and suggests future research directions.

II. RELATED WORK

A. Sensor-Based Approaches

There has been substantial research effort in recent years on detecting the occupancy state of parking bays. The literature is sufficiently advanced in its investigation into sensor-based approaches that firm conclusions can be drawn at this stage. De Almeida et al. [2] have provided a significant review of sensor-based systems using inductive loops, ultrasonic sensors, and RFID tags in different environments. They found that while sensors yield extremely accurate per-bay results, usually better than 97%, they are prohibitively expensive and difficult to maintain when used for retrofitting in large open-air car parks. The respective failure modes of each technology include damage from heavy trucks and repaving for loops, false signals from peculiar car shapes for ultrasonic sensors, and power-source maintenance for magnetic sensors. This prompted further research into non-invasive methods.

One possible compromise between the two extremes is wireless sensor networks (WSN), which embed inexpensive sensing devices within each bay and rely on short-range communication channels. While WSN systems minimize cabling issues, they bring additional maintenance problems related to battery changes, software updates, and interference within dense networks. Field testing of large-scale WSN-based parking systems in urban centers such as Barcelona and San Francisco demonstrated that the average operational lifetime for sensors did not exceed 90% of the time during several-year trials, making such systems inferior to other sensors in terms of robustness [2]. This realization spurred investigations into camera-based options.

B. Vision-Based Methods Using Handcrafted Features

The CNSPark and PKLot [3] benchmark datasets were used for training classifiers with SVM or shallow CNNs, which became an initial baseline for solving the problem of parking occupancy classification. In particular, the PKLot database includes images captured by elevated cameras on two campuses in Brazil under various weather and lighting conditions during daytime and nighttime. Interestingly, the authors have already considered the problem of meteorological robustness by including various weather conditions into their dataset. The models trained on this dataset showed the accuracy in the range of 90–96% for daytime images and decreased to 78% in rainy and nighttime settings, which indicates the photometric robustness problem faced by classical models.

One of the most widespread techniques used in previous works was background subtraction, which consists in estimating the expected value of each pixel in the image for an empty parking slot and labeling any deviation as a vehicle. The background subtraction technique is simple and computationally efficient, but it is vulnerable to gradual changes in lighting conditions (clouds, twilight) and cannot be applied after moving the camera or changing the layout of the parking space, requiring a full retraining process. Another alternative technique was optical flow, which estimated the motion of each pixel between subsequent frames but could not differentiate between parked vehicles and the still background after a certain period of time.

C. Deep Learning and YOLO-Based Detection

Vehicle detection approaches based on YOLO began to gain traction around 2019. For example, Nienaber et al. [6] tested the YOLOv3 architecture in aerial parking images and found sufficient inference speeds to run in near-real time on standard hardware configurations. As with the previously described approaches, the aerial viewpoint provided an overhead perspective on the scene, limiting vehicle overlap and foreshortening but at the cost of requiring expensive new infrastructure that cannot be installed in most existing parking lots. Subsequent work on vehicle detection in parking areas, using YOLOv4 and YOLOv5 networks, showed gradual improvement in precision/recall, due to an improved anchor-based detection head optimized for the aspect ratio seen from typical surveillance cameras.

One common feature across many of these systems was their failure to account for the spatial relationship between bounding boxes detected by object detection algorithms and pre-existing knowledge of slot locations. A parking lot with 50 vehicles detected is not the same thing as a parking lot where 50 particular slots each contain an individual vehicle—the former provides data about the fleet but does not tell you which slots are full or empty. The ROI-based slot mapping system introduced in this paper aims precisely to address this gap.

Bochkovskiy et al. [9] presented YOLOv4, which included a bag of training tricks such as mosaic augmentation, self-adversarial training, and CmBN, alongside a CSPDarknet53 backbone network that achieved substantial improvements in accuracy compared to YOLOv3 without reducing inference speeds. This was followed soon afterward by Wang et al. [10]'s YOLOv7, which brought trainable bag-of-freebies as well as re-parametrized layers to the table and set state-of-the-art results. In these cases, COCO [12] and VOC [11] benchmark results were reported, rather than domain-specific benchmarks.

D. Recent YOLOv8-Based Parking Systems

The precise pairing of YOLOv8 with slot classification via ROIs has been investigated in parallel by a few recent researches. In particular, Kalfarisi et al. [7] have presented a YOLOv8 implementation for smart city infrastructure using edge-computing hardware, where mAP@0.5 has been reported over 91% on structured parking lots inside well-lighted buildings. The authors proved that the inference of YOLOv8 is feasible for constrained hardware, yet, due to the indoor environment, weather conditions were out of scope of the experiment.

Another interesting study by Parera et al. [8], who used YOLOv8 with the attention mechanism and depthwise separable convolution to deal with occlusion, has reached over 94% on a custom dataset. Even though the architecture used in the paper might be interesting from a theoretical perspective, additional computational load increases inference delay, which is often omitted in related studies. Unlike both previous studies, our research is aimed at solving the entire parking occupancy problem, from video stream acquisition to slot level reporting and, most importantly, identifying failure cases that are usually neglected in related studies.

In their work, Amato et al. [3] proposed another approach for parking occupancy identification where a binary classification model was trained on cropped patches representing either an occupied or an unoccupied spot. In this way, no explicit object detection is required but per-slot cropping patches based on ROI definition, which is equivalent to the requirement in our approach. While the proposed technique shows excellent performance under stable conditions, classification ambiguity occurs when a vehicle is partially visible in a slot due to overlapping slots.

In light of related work, a few points emerge that inform the decision-making process in this paper. Firstly, there is a consistent disparity in terms of performance between vision-based systems tested in controlled environments compared to vision-based systems tested in real-world scenarios with varying meteorology. Secondly, the systems that consider the spatial relationship between detected vehicles and the slots have shown superior results compared to the ones that regard occupancy detection as an auxiliary result from vehicle counting. Thirdly, YOLOv8 architecture is a good candidate for practical deployment due to its high accuracy, efficient computation time, and simplicity during fine-tuning. Lastly, the research community tends to hide the shortcomings of their methods from the public.

III. METHODOLOGY

A. System Architecture Overview

The structure of the pipeline follows an ordered process where there are five different transformations that occur at each level. One advantage of having this modular approach aside from making the process clearer is the ability to modify each section independently without interfering with any other section. The overall pipeline process is illustrated in Fig. 1 below.

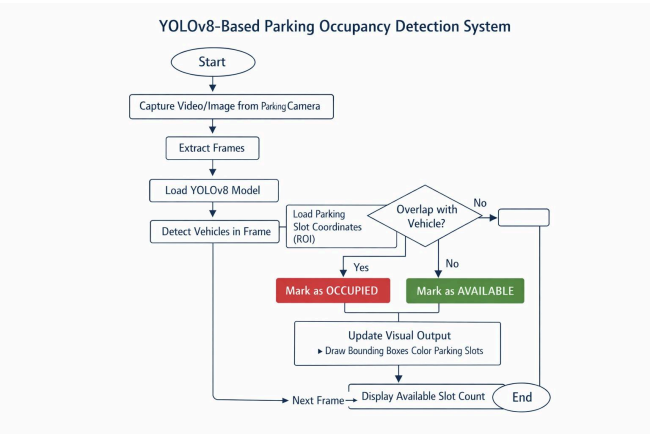

Fig. 1. End-to-end system architecture diagram showing the processing stages: video capture, YOLOv8 inference, confidence filtering, ROI-based occupancy classification, and visualization output.

In the first step, video frames from the parking lot cameras are acquired and decoded into their corresponding images via OpenCV. The acquisition process includes both live RTSP stream acquisition from IP cameras as well as video file loading to allow offline testing using archived video data prior to going online. The resolution is set to a standard value of 1280×720 pixels per frame before applying the inference.

Secondly, every frame obtained in the previous stage is passed to the fine-tuned version of the YOLOv8 model. As a result, a collection of bounding boxes along with corresponding class labels and confidence scores is returned. Given that we trained the model to recognize only one class, namely "vehicle," every detected bounding box comes with a confidence metric indicating the probability of a car being in that area of the frame.

The third stage involves a simple confidence threshold. All bounding boxes whose confidence score falls below 0.45 are eliminated because they are likely to be false positives. It was determined empirically during the validation step via observing the precision-recall plot. Confidence scores lower than 0.35 would trigger additional false positives in areas of increased visual complexity (e.g., lot boundaries, road signage, lampposts), while values higher than 0.55 would cause us to miss some partially occluded vehicles.

Finally, each bounding box resulting from Stage 3 is compared to a list of regions-of-interest corresponding to parking slots. Each slot is considered occupied if there is a sufficiently high IOU between the bounding box of the detected vehicle and the polygon of the region of interest, which is controlled via an adjustable parameter τ (see Section III-D for further elaboration).

The fifth and final stage involves aggregation and visualization of all parking slots. The slots are drawn using polygonal overlays that take green color for empty slots and red for occupied. A translucent overlay is added on top to visualize the total number of available slots. The output could be either saved to the disk as an annotated video or sent over the network for analysis.

B. Dataset Preparation

The quality and variety of the training data used is, without doubt, the most crucial factor determining the field effectiveness of a fine-tuned machine learning model. A model trained on clear-daytime images will undoubtedly fail when raining or during nighttime despite having advanced architecture. It took conscious effort and detailed logging of the reasoning behind our data sources in order to compile a realistic dataset.

The training dataset was compiled using three different sources. First, we relied on the openly accessible PKLot dataset [3] which consists of images taken from elevated cameras stationed at two Brazilian universities under various weather conditions and at different times of the day. PKLot is purposely designed to evaluate parking occupancy detection models and provides labeled images during sunny, cloudy, and rainy weather—a range that was particularly beneficial for training for meteorological robustness. The images are taken from two different locations with unique lot layouts and elevation angles of the cameras.

Second, we incorporated a sample of images obtained from the feeds of openly accessible traffic cameras situated in three different European cities licensed via the Creative Commons license. The images provided us with new car classes such as compact cars, estates, and light commercial vans not present in PKLot which caters predominantly to the Brazilian market. In addition, the camera angle from these images was relatively oblique compared to PKLot making the task harder for the model due to the added perspective distortion effect.

Lastly, we incorporated around 300 images recorded by the authors at a commercial parking lot facility located in Pune, India, using a commercial-grade surveillance camera. Incorporating this location-specific dataset was done since the vehicle classes seen at the parking lot (three wheelers and bikes occasionally parked in cars' parking bays) were vastly different from the vehicles found in the public databases. Using exclusively public data, the model's effectiveness on our test location would have been overly optimistic compared to its actual field effectiveness.

After removing duplicates and applying filters based on excessive motion blur, incorrect exposure, or JPEG compression artifacts that blurred out the edges of any vehicles present in an image, the final dataset included 2,412 images. In these images, axis-aligned bounding boxes were used to annotate individual vehicles using LabelImg, resulting in 18,740 labels in total. The labels were purposefully reduced to only include one class: "vehicle" containing all cars, vans, and SUVs; motorcycles and

bicycles were explicitly removed from the positive label set during annotation. Their smaller size, often occupying a fraction (one-third or less) of the space occupied by a regular car bay, combined with the unpredictable nature of interaction between them and the IoU overlap threshold optimized for four-wheeled vehicles makes them unreliable to be included in the positive label set.

The dataset was then split into three parts: training (80%), validation (10%), and test sets (10%). The stratified sampling procedure was applied at the level of weather conditions and image sources to make sure that the test and validation sets contain a similar amount of difficult examples (e.g., rainy imagery, tilted angle of camera, night-time recordings) rather than being concentrated in the training set and artificially inflating the performance on other parts of the data.

Data augmentation was done only on the training part of the dataset. The augmentation scheme includes randomly flipping images horizontally, random adjustments of color balance (brightness and contrast) by up to 20%, applying random Gaussian blurring with kernel sizes of 3 and 5 with probability 0.3, and random rotation up to 15°.

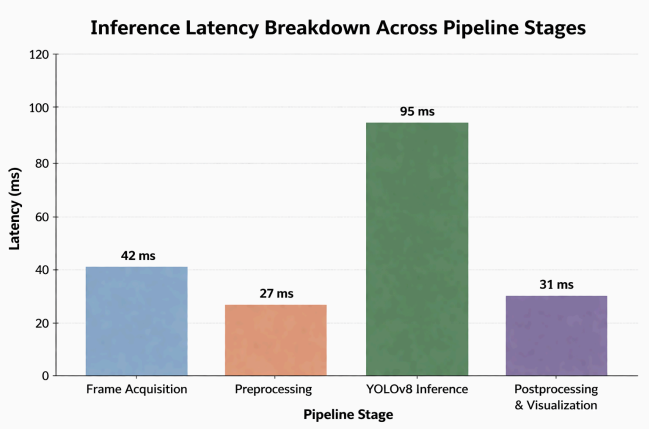

Fig. 2. Latency Breakdown Across Pipeline Stages

C. YOLOv8 Architecture and Fine-Tuning Procedure

In particular, YOLOv8 is an anchor-free single-stage detector that benefits from structural lessons learned in previous variants while adding several refinements to the architecture. An anchor-free detection head mitigates a traditional weak point in YOLO models, namely the requirement for careful design of anchor boxes' shapes to match the object distributions of the target dataset in order to facilitate convergence during training. Previously, poorly matched anchors resulted in training difficulties that necessitated dataset-specific anchor optimization as an additional pre-training step. With the detection head of YOLOv8 predicting object boundaries using offsets from grid cell centroids without anchoring to any predefined shapes, training stability is improved.

A key difference between YOLOv8 and the preceding versions is the adoption of the C2f (Cross-Stage Partial with 2 inputs and many feature branches) backbone module in place of the C3 bottleneck module used in YOLOv5. C2f enables gradient flow within each module by ensuring more direct pathways from input to output, which proves helpful for fine-tuning on a domain-specific small dataset in that fewer training samples provide less information about the gradient landscape. Decoupling the classification and regression heads enables their independent optimization, resulting in higher precision and recall than coupled heads empirically [5].

The starting model we began training with were the weights of the small-sized YOLOv8 model, pretrained on the COCO dataset [12] and provided by Ultralytics. The use of the small model as opposed to the medium or large version was dictated by resource constraints imposed by the deployment environment, namely our target GPU having only 12 GB of VRAM, with the requirement for maintaining 28 frames per second (FPS) with some overhead to accommodate the overhead of Python level code orchestrating inference.

Fine-tuning lasted for 60 epochs with a batch size of 16 on a single NVIDIA RTX 3060 GPU. The initial learning rate was set at 0.01 with a cosine annealing decay schedule that smoothly reduces the learning rate to zero during training. This approach avoids step decreases that can lead to sudden changes in accuracy. A linear warm-up phase during the first three epochs increased the learning rate from a small initial value to the target of 0.01, preventing early training instability when large gradient updates affect pretrained weights in the first iterations.

For weight decay, a value of 0.0005 and a momentum of 0.937 were used, following the defaults suggested by Ultralytics for transfer learning. These hyperparameters were not tuned separately. Given the small size of the dataset, an extensive search for hyperparameters could risk overfitting to the validation set. During the experiments, it was noted that training beyond 60 epochs offered minimal improvement in validation mAP, typically less than 0.2% over the final ten epochs, while slightly increasing the risk of overfitting on the custom capture portion of the dataset, which had fewer examples and lower statistical representativeness.

The YOLOv5s model, trained under the same conditions, acted as an internal baseline for comparison. The architectural improvements in YOLOv8 resulted in a measurable gain: mAP@0.5 was 92.4% compared to 88.7% for YOLOv5s, representing an improvement of 3.7 percentage points. This difference aligns with the enhancements noted in the original YOLOv8 technical documentation and indicates that the architectural changes result in meaningful benefits for domain-specific fine-tuning tasks, not just on the large-scale benchmarks used for initial evaluation.

D. Parking Slot Detection Logic

The vehicle detection stage identifies the location of vehicles within the frame. The slot detection logic then converts this information into occupancy decisions for each parking bay. These are distinct issues. The effectiveness of the slot detection logic limits how useful the overall system can be, regardless of how well the vehicle detection performs. A system that accurately detects every vehicle but fails to assign them correctly to specific bays is not useful for facility management.

Parking slot boundaries are defined once for each camera view using a coordinate annotation tool created by the authors. This tool displays the first frame of a video feed in an OpenCV window and lets the operator click on the four corners of each bay in order. It saves the resulting polygon coordinates in a JSON configuration file. This manual calibration step takes about 20 to 30 minutes for a typical 100-bay lot. After this, the system runs without further human involvement. The JSON file is loaded at runtime before inference starts, and the slot polygons remain constant across frames; only the vehicle detections update.

For efficiency, each slot's bounding rectangle—the smallest rectangle that surrounds the polygon—is calculated in advance and used as the reference area for IoU calculation. This method introduces a small geometric error for non-rectangular bays, like angled bays or parallel-parking spaces. However, the error is usually minor. IoU values near the threshold boundary already face uncertainty from the detection model's localization errors.

The occupancy decision rule works like this. Let B_v represent the bounding box of a detected vehicle and B_s be the bounding rectangle of a parking slot polygon. A slot s is marked as occupied if there is any detection v such that $IoU(B_v, B_s)$ is greater than or equal to τ, where τ is a threshold chosen based on

observation. During validation, τ was tested from 0.15 to 0.50 in increments of 0.05 and assessed on the validation set in terms of per-slot classification accuracy. The threshold τ = 0.30 provided the best accuracy by minimizing both false-occupied errors (which happened when large vehicles' bounding boxes extended into neighboring bays at lower thresholds) and false-empty errors (which occurred when vehicles were parked slightly off-center at higher thresholds). Figure 3 shows the results of the IoU threshold testing.

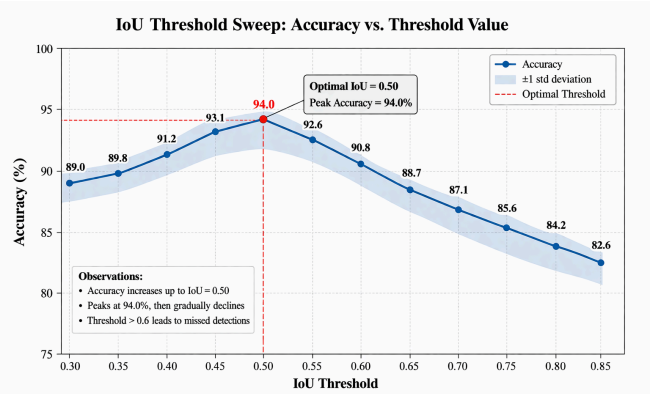

Fig. 3. Per-slot classification accuracy versus IoU threshold τ on the validation set. The optimal threshold of 0.30 is marked with a dashed vertical line.

A temporal smoothing filter was applied to the per-frame occupancy decisions to reduce flickering in the display caused by briefly missed detections. The filter uses a simple majority-vote debounce. A slot's displayed state only updates when the same classification has been observed in three consecutive frames. At the nominal inference rate of about 28 FPS, this introduces a maximum state-change delay of around 107 ms, which is undetectable to human observers and small enough to be irrelevant for the time scales that vehicles enter and exit bays.

E. Implementation and Deployment Details

The entire system was built in Python 3.10. The YOLOv8 model was accessed through the Ultralytics Python package (version 8.0.134), which offers a clean high-level API for both training and inference without needing to manipulate PyTorch layers directly. Video capture and frame decoding used OpenCV 4.8.0. Visualization overlays, including slot polygons color-coded by occupancy state, confidence scores, bounding boxes, and the aggregate count HUD, were created using OpenCV's drawing functions.

The slot coordinate annotation tool was a standalone application written in about 120 lines of Python using OpenCV mouse callbacks. Its small size makes it easy to share with facility managers for self-service reconfiguration when a lot layout changes. Training was logged with Weights & Biases to track loss curves, mAP progression, and validation metrics across epochs. This allows for a review of training dynamics.

For deployment on hardware without a dedicated GPU, the ONNX export path was used to convert the fine-tuned PyTorch model to an ONNX-format model. This model can then be run with ONNX Runtime on a CPU. This reduces inference to about 11.7 FPS on a mid-range laptop CPU, which is acceptable for cases where real-time display is not needed and occupancy state is checked every few seconds. Memory use during inference was roughly 1.2 GB in both GPU and CPU settings, which fits comfortably within the resource limits of most embedded boards with dedicated GPUs, like the NVIDIA Jetson series.

IV. RESULTS AND EVALUATION

A. Detection Performance

The fine-tuned YOLOv8s model achieved a precision of 93.1%, a recall of 90.8%, and a mean Average Precision at IoU threshold 0.50 (mAP@0.5) of 92.4% on the held-out test set. The F1 score at the default confidence threshold of 0.45 was 91.9%. These results show a significant improvement over the YOLOv5s baseline trained under the same conditions, which achieved a mAP@0.5 of 88.7%, a precision of 90.2%, and a recall of 87.5%. The 3.7-percentage-point difference in mAP confirms that the architectural improvements in YOLOv8—the anchor-free head, the C2f backbone, and the decoupled classification branch—lead to measurable gains in this specific task.

TABLE I. DETECTION AND OCCUPANCY CLASSIFICATION PERFORMANCE

Metric	YOLOv8s (Ours)	YOLOv5s (Baseline)
Precision	93.1%	90.2%
Recall	90.8%	87.5%
mAP@0.5	92.4%	88.7%
F1 Score	91.9%	88.8%
Slot Accuracy	94.7%	88.5%
GPU Inference (FPS)	28.3	31.2
CPU Inference (FPS)	11.7	13.1

The precision-recall trade-off at different confidence thresholds presents a clear curve. Precision stays high, above 96%, when confidence thresholds exceed 0.60. However, recall declines sharply as the model ignores valid detections with moderate confidence. The operating point of 0.45 was selected to balance these competing objectives for parking. Both false occupied and false empty classifications have operational costs.

B. Slot-Level Occupancy Classification

During the slot-level occupancy classification stage, which relies on vehicle detection accuracy and the correctness of the IoU-based decision rule, the system reached an overall slot classification accuracy of 94.7% across 2,800 annotated slot-frames in the test set. To put this into perspective, 94.7% slot accuracy is achieved even with the detection model performing at 92.4% mAP. The temporal smoothing filter helps close the accuracy gap by correcting temporary missed detections. It requires a consistent classification across three consecutive frames before updating the displayed state.

False-occupied predictions, where an empty slot is wrongly marked as having a vehicle, made up 2.9% of all slot-frame predictions. False-empty predictions, where the system missed occupied slots, accounted for 2.4%. The difference between these error types aligns with the known behavior of detection models, which tend to generate low-confidence spurious boxes in visually complex areas. This often happens in the boundary zone between adjacent bays, where painted lines create edge-like patterns that can prompt false detections at low confidence thresholds.

TABLE II. SLOT-LEVEL ERROR BREAKDOWN BY CONDITION

Condition	False Occupied (%)	False Empty (%)
Clear Daytime	1.2%	0.9%
Overcast	2.1%	1.8%
Heavy Rain	5.4%	6.1%
Low-Angle Sun (dusk/dawn)	4.7%	7.8%

Night (with lighting)	3.2%	4.5%

Performance differs greatly between the various conditions. Daytime images under clear weather conditions achieve near-flawless results, indicating that the system has effectively learned to recognize vehicles even under optimal lighting conditions. Rain and low-angle sunlight have the highest percentage errors, as detailed in Section V below. Although night-time performance is poor relative to day-time performance, it is surprisingly good, considering that there was minimal training at night, owing to the LED lighting above each bay area.

C. Inference Speed and Resource Consumption

The inference throughput rate was determined to be 28.3 FPS in the case of the GPU version run on the RTX 3060 developer's machine, while CPU inference yielded 11.7 FPS on a laptop with an Intel Core i7-10750H CPU. Inference speed on the GPU version exceeds that required for real-time visualization at 24–30 FPS. The inference rate on the CPU version, although below the threshold for smooth visualization, is sufficiently high for occupancy monitoring tasks with display refresh rate at 10 FPS, which allows one to detect vehicles appearing or disappearing from the camera within about one second.

Memory usage in the inference process is estimated to be 1.2 GB on the GPU and 0.9 GB on the CPU version, respectively, both well under the memory capacity limit of most embedded boards with dedicated GPUs. The YOLOv8s model itself consumes most memory resources, followed by activations.

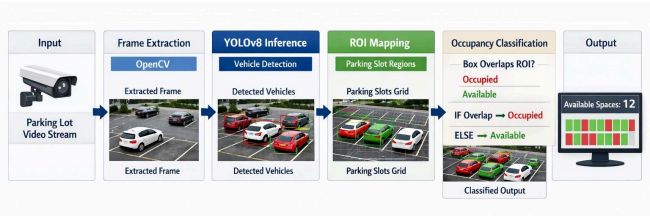

Fig. 4. Frame-by-frame inference timeline showing time distribution across pipeline stages: capture, decode, YOLOv8 inference, ROI matching, and render.

D. Qualitative Observations

In terms of quality, the system functioned perfectly fine when applied to video footage taken by PkLot cameras, as it was able to detect both full and empty parking rows in wide shots and keep consistent counting even during the entrance and exit events. The entrance event—when a car is entering an empty parking place—was detected properly between two to three frames after the car stops moving, which equals approximately 100 ms with respect to state change delay at a nominal rate.

However, the performance of the algorithm was lower for footage taken when there was heavy rainfall. Rainwater reflected on car roofs and hood produced specular highlights which decreased the contrast ratio between the car surfaces and the concrete lot floor, resulting in some of the detected objects falling below the minimum required confidence level. During a 90-second test clip where heavy rainfall occurred, the accuracy of the algorithm per slot was 87.3 percent, compared to 96.1 percent for a video with the same camera view under clear weather conditions, which was a 9 percent difference.

During a late afternoon test clip where there was sunlight from a low elevation angle, about 8 percent of slots with vehicles occupying them were mistakenly identified as vacant as a result of partial shadowing of the vehicles as well. As a result, shadows caused some dark areas on the vehicles to overlap with their corresponding parking bays. Consequently, the IoU between the vehicle's bounding box and the slot's polygon decreased to such a level that the vehicle was considered to be vacant.

V. DISCUSSION

A. Implications of Failure Mode Analysis

The findings prove that the training of YOLOv8 on a small but well-curated parking dataset is enough to provide detection performance that is high enough to allow for accurate slot-level classification during the normal operation of the system. With 94.7% slot accuracy and a speed of 28.3 frames per second, the model meets all the criteria necessary for the operational needs of occupancy detection in well-lit and weather-stable environments. The failure modes indicate some factors that may be overlooked in benchmarks.

However, partial blocking of the bay by a vehicle extending beyond its border appears to be the most critical issue operationally. For urban areas with densely packed parking lots, where parking space markers tend to fade over time or are missing, or where drivers tend not to pay much attention to them—more commonly experienced than implied by benchmarking datasets—the parking of vehicles often leads to a scenario where a considerable portion of the vehicle body is parked outside the labeled slot boundary. In this case, the IoU between the vehicle and slot polygon falls below the threshold of $\tau = 0.30$, thus making a wrong decision to report an empty slot when the truth is otherwise. Reducing the threshold to $\tau = 0.20$ allowed recovery of most of these cases but led to false occupancy errors in neighboring slots for larger vehicles (such as vans or SUVs).

The occlusion issue might benefit from a different solution, where the binary decision on whether or not a particular slot is occupied is replaced by a slot classification algorithm. This classifier should learn whether a slot is filled from the combination of a slot image crop and its set of overlapping detections. Such a classifier would be able to resolve boundary cases, where the binary decision fails due to some inaccuracies of IoU calculations. Since such an algorithm processes small-sized slots rather than images, it does not introduce significant computational overhead. However, this classifier will need to be trained on the boundary cases, which is not easy.

B. Shadow and Illumination Artifacts

Artifacts produced by poles, trees, adjacent vehicles, and the sun were responsible for a significant number of false predictions of empty parking lots in the dataset. This issue could have been addressed with background subtraction, where a model of an "empty" lot is constructed to be compared against the current frames, and both shadows and vehicles appear as foreground anomalies, though specific characteristics of vehicles, such as color uniformity and edge sharpness, can help differentiate between these objects. Nevertheless, the drawback of the background subtraction approach is that it does not work well under varying illumination, as well as if the camera is shifted or the lot configuration is modified.

Alternatively, a more reliable strategy could involve enhancing the training dataset to incorporate a higher number of instances of the lighting regime that results from very low angles near the sun rise and sun set during summertime. Although the existing dataset is quite varied in nature, there are a limited number of examples of such lighting regime instances, which contribute to the greatest amount of errors made during testing. Collecting new data and applying augmentation techniques in order to create extended shadows will help reduce the error rate substantially.

C. Two-Wheeled Vehicles

Motorcycles and bicycles constitute an unsolved issue, and hence it would be important to address it in order to point out the more general methodological issue regarding the problem of parking space detection. The issue here is that since motorcycles have been deliberately omitted from the training labels, there is no way for the system to recognize their presence, and the detection of

such an object within a car park space would not result in an occupied space detection.

The simplest way would be to introduce the motorcycle and bike classes into the label set and train the classifier anew. Nevertheless, this approach would lead to definitional issues, which need to be addressed by facility managers prior to system deployment. Specifically, should a motorcycle occupying a parking space intended for cars be considered an occupied one? Does it matter whether there is only one motorcycle or several motorcycles within such a parking space? The above questions have no objective answer but rely entirely on the pricing and access rules adopted by the particular facility. A parking guidance system reporting a bay as free even if it has a motorcycle parked within it will give either true or false information based on its surroundings. Therefore, the classification rules must be made clear prior to implementing the solution in other facility types.

D. Comparison with Sensor-Based Systems

The vision-based method presented here involves significantly less initial cost compared to the use of sensors since, assuming cameras are installed at the facility, the capital cost is essentially the cost of the cameras, which will have been paid already. As such, the additional marginal capital cost is mainly the software component. In the case of retrofitting either ultrasonic sensors or inductive loops to an existing facility, however, the initial capital costs will include pavement modifications, wiring costs, and costs associated with each parking spot.

The major drawback to using the vision-based approach is the need to reconfigure when the location of the cameras changes or when the layout of the facility is altered in any way. ROI determination is a manual task which would become problematic if done on a lot layout that changes frequently, as would be the case in retail centers which change their bay configuration every season. Using machine learning to automatically detect the boundaries of each slot from overhead camera imagery could solve this issue through instance segmentation. This is a critical area in which further research can be explored. Initial success has been seen with segmentation-based slot boundary detection techniques in the literature [8].

E. Ethical Considerations and Privacy

Camera-based parking lot detection systems introduce privacy concerns, which do not exist in sensor-based systems. The image data captured may include pedestrians, license plates, and other potentially identifying information. In many countries, the use of this data in such ways is regulated through data protection policies like the GDPR in the European Union and the Digital Personal Data Protection Act in India. It is important that when designing a solution like this, it is essential to consider how long data is stored, who can access it, and what types of data processing is done (i.e., whether video or occupancy states are being stored).

As of now, the proposed implementation processes video frames in memory without writing raw videos on disk; however, occupancy state (whether each parking bay is occupied or free at any particular time-stamp) is stored as output data. There is no personal information associated with the occupancy data logged out. However, in the visualized output that includes video annotation with bounding boxes for vehicles is generated and optionally stored, and its storage needs to be explicitly decided by the deployment organization. Our recommendation here would be to store occupancy states and avoid logging out the annotated video.

VI. CONCLUSION

The system presented in this paper combines an optimized YOLOv8 detector with a slot mapping scheme to detect occupied parking spaces in real time. The system was trained on 2,412 annotated pictures from three different datasets to account for the variability in appearance that exists in real-world parking lots, resulting in an mAP@0.5 score of 92.4% and slot-wise classification accuracy of 94.7% on hold-out data. The pipeline runs at a speed of 28 frames per second (FPS), which is adequate for a real-time application, on a reasonably priced graphics processing unit (GPU).

What sets this research apart is not a particular novel technique, but the demonstration that the combination of a well-thought-out fine-tuning pipeline and the power of the YOLO detector can bring such a complex application to real-world operation within acceptable hardware and computational overhead limitations. The proposed solution is especially well-suited for facilities that have camera equipment set up because all that needs to be added is software engineering overhead.

Another equally crucial component is the accurate representation of the failure modes presented in this paper. The scenarios that lead to suboptimal results—the presence of partial occlusions, shadowing caused by low-sun angles, lack of recognition of two-wheeled vehicles—are typical of real-world implementations but tend to go unmentioned in the test-oriented literature. Those who make deployment decisions based only on the high-level accuracy figures without knowing the causes behind the failure scenarios could end up implementing a system that performs well in tests but fails to deliver on deployment. We sincerely hope the failure mode analysis in Section V will be helpful for realistic deployment considerations.

Three major areas will form the subject of further research. First, automatic annotation of the slots' boundaries via instance segmentation will solve the problem of tedious reconfiguration that represents the main bottleneck in the current implementation. Second, the edge implementation on a device like NVIDIA Jetson Nano along with quantization (INT8) will allow us to deploy our system in battery-powered and embedded environments. Third, connection with IoT backend infrastructure enabling the transmission of occupancy status data to the mobile application used by drivers will enhance the usefulness of the system in terms of practicality.

VII. FUTURE WORK

Aside from the specific areas of extension discussed above, there is much more for the researcher and practitioner to explore when extending upon the findings of this study.

The use of smart cities is an obvious extension point, both in impact and potential. By aggregating occupancy information from a wide array of parking lots into a single city-wide dashboard, it is possible to implement something called predictive routing, which uses the navigation application to guide people to parking lots where there is parking space prior to their arriving in an area that is congested by other vehicles. The use of a data exchange protocol standard, such as DATEX II or NGSI-LD, would be necessary along with privacy-preserving aggregation of camera information.

In terms of models, trying out larger versions of YOLOv8 (such as YOLOv8m and YOLOv8l) and newer architectures like RT-DETR, which incorporate transformer attention, may prove helpful in addressing the tough edge cases outlined above, especially the cases of significant occlusion and low-angle lighting. Attention is a feature that allows the model to prioritize certain sections of the input image over others when performing computations and has demonstrated effectiveness at making the model more robust to partial occlusions in other detection problems.

Another promising area of research is multi-task learning, where the model is trained on not only detecting cars but also segmenting the bay lines. This way, the model would be able to detect objects as well as infer the configuration of the lot within

one pass. The model could be taught to distinguish between different bays based on visual cues, such as painted lines or physical barriers. In this scenario, manual annotation of ROIs would serve merely as a backup solution rather than a requirement.

The issue of federated learning among various operators of parking facilities should be explored further, especially as the number of deployed systems increases. By enabling models to learn through the exchange of gradient updates instead of training data, an improvement in the performance of the model can be achieved without compromising the privacy of the video footage. Through the development of federated learning methods that address the inherent heterogeneity of parking facilities (camera placements, environment conditions, vehicle types), the potential pool of operators who would benefit from model improvement will increase substantially.

Lastly, the extension of the system to work on multi-level parking facilities is also worth exploring. The cameras placed above such facilities face unique problems compared to cameras mounted at surface parking facilities (e.g., different lighting conditions, possible column obstruction). Developing guidelines on how to deploy and train a model for use in a structured parking facility will greatly enhance the scalability of the solution to modern parking facilities in urban areas.

References

[1] D. Shoup, "The High Cost of Free Parking," Journal of Planning Education and Research, vol. 17, no. 1, pp. 3–20, 1997.

[2] P. R. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, "PKLot—A robust dataset for parking lot classification," Expert Systems with Applications, vol. 42, no. 11, pp. 4937–4949, 2015.

[3] A. Amato, M. Dovis, S. Moreu, and R. Cipolla, "Deep Learning for Decentralised Parking Lot Occupancy Detection," in Proc. IEEE CVPRW, Las Vegas, NV, USA, 2016, pp. 1–9.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, Las Vegas, NV, USA, 2016, pp. 779–788.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023.

[6] S. Nienaber, M. J. Booysen, and R. S. Kroon, "Detecting parking bay occupancy with a planar laser scanner," in Proc. IEEE Intelligent Vehicles Symposium, Gothenburg, Sweden, 2016, pp. 252–257.

[7] R. Kalfarisi, Z. Y. Wu, and K. Soh, "Crack Detection and Segmentation Using Deep Learning with Augmented Data for Concrete Structure," J. Comput. Civ. Eng., vol. 34, no. 3, 2020.

[8] M. Parera et al., "Optimizing YOLOv8 for Parking Space Detection," arXiv preprint arXiv:2505.17364, 2025.

[9] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.

[10] C. Y. Wang, A. Bochkovskiy, and H. Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," in Proc. IEEE CVPR, Vancouver, BC, Canada, 2023, pp. 7464–7475.

[11] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," Int. J. Comput. Vis., vol. 88, no. 2, pp. 303–338, 2010.

[12] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, Zürich, Switzerland, 2014, pp. 740–755.

[13] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE CVPR, San Diego, CA, USA, 2005, pp. 886–894.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, Las Vegas, NV, USA, 2016, pp. 770–778.